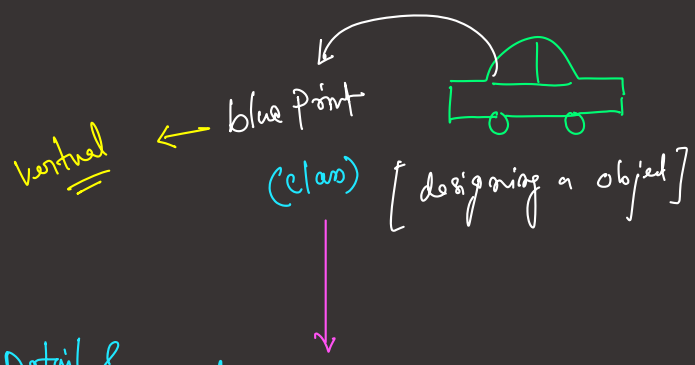


Oops

Object Oriented Programming.



Principles of Oops

1. Abstraction [hiding internal detail & showing only required things] Real world Entity
2. Encapsulation [everything in a single box] (Object)
3. Inheritance [old feature with some extra new feature]
4. Polymorphism [Multiple things by a single name]

class Car {

① Properties { int windows
int wheels
int seats } Private

② behaviour [method] Public
↳ it will utilized the properties

1. Specialization [Inheritance]
2. Generalization [Polymorphism]

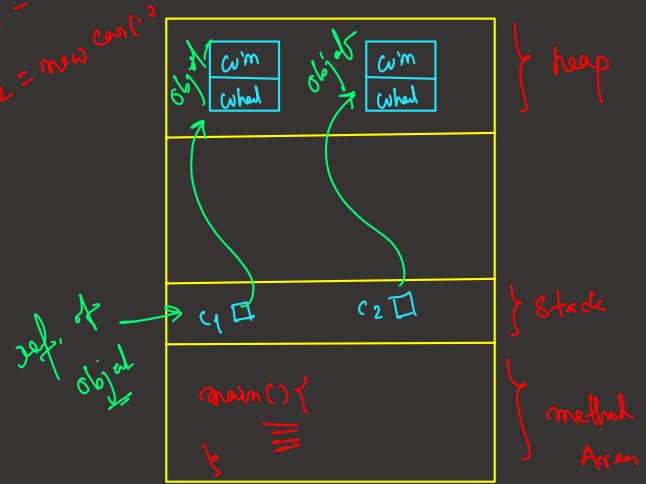
* toString()

ArrayList arr = new ArrayList(1, 2, 3, 4, 5)

arr → [1 | 2 | 3 | 4 | 5]

System.out.println(arr) →
1, 2, 3, 4, 5

Car c1 = new Car(4, 4)
Car c2 = new Car(2)

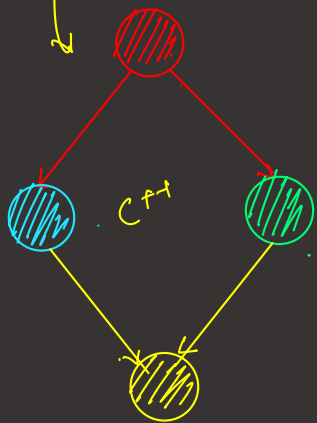


java Access specifier

	default	private	protected	public
same class	yes	yes	yes	yes
same package subclass	yes	no	yes	yes
same package non-subclass	yes	no	yes	yes
different package subclass	no	no	yes	yes
different package non-subclass	no	no	no	yes

2 Inheritance

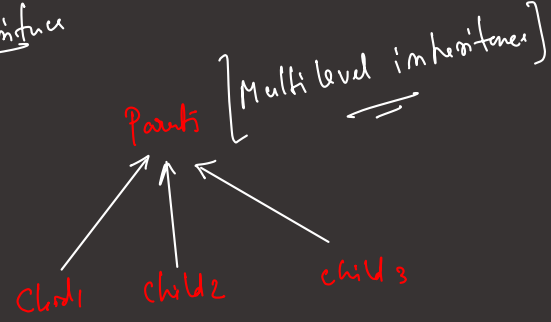
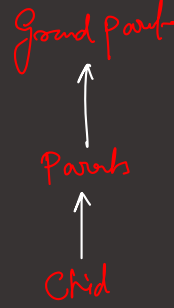
Diamond Problem



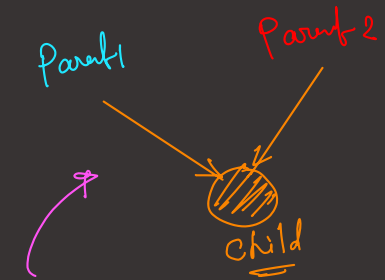
Single Inheritance



Hierarchical Inheritance



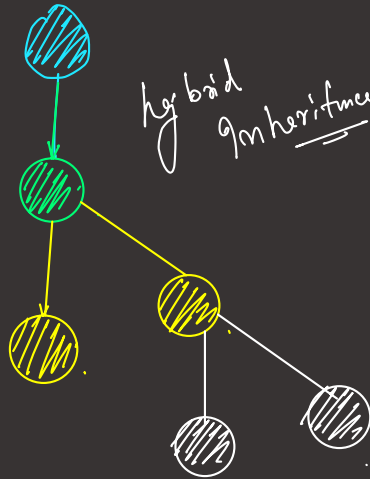
Multiple Inheritance



not in java (class)

using interfaces it is possible

hybrid Inheritance



day 12

class Circle {

private int r;

Circle (int r)
this.r = r;

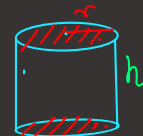
{

public double area() { ... }

public double perimeter() { ... }

}

circle
↑
cylinder



class Cylinder extends Circle {
private int h;

Cylinder (int r, int h)
super (r);
this.h = h;

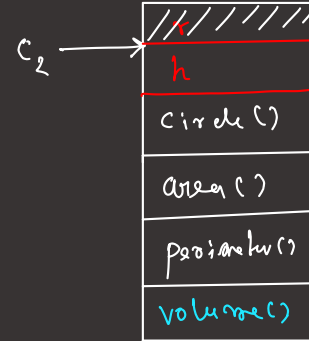
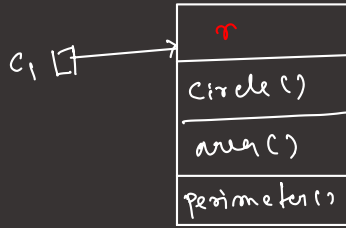
{
public double volume() { ... }

}

reference
 object
 Circle c1 = new Circle(5);
 Cylinder c2 = new Cylinder(10);

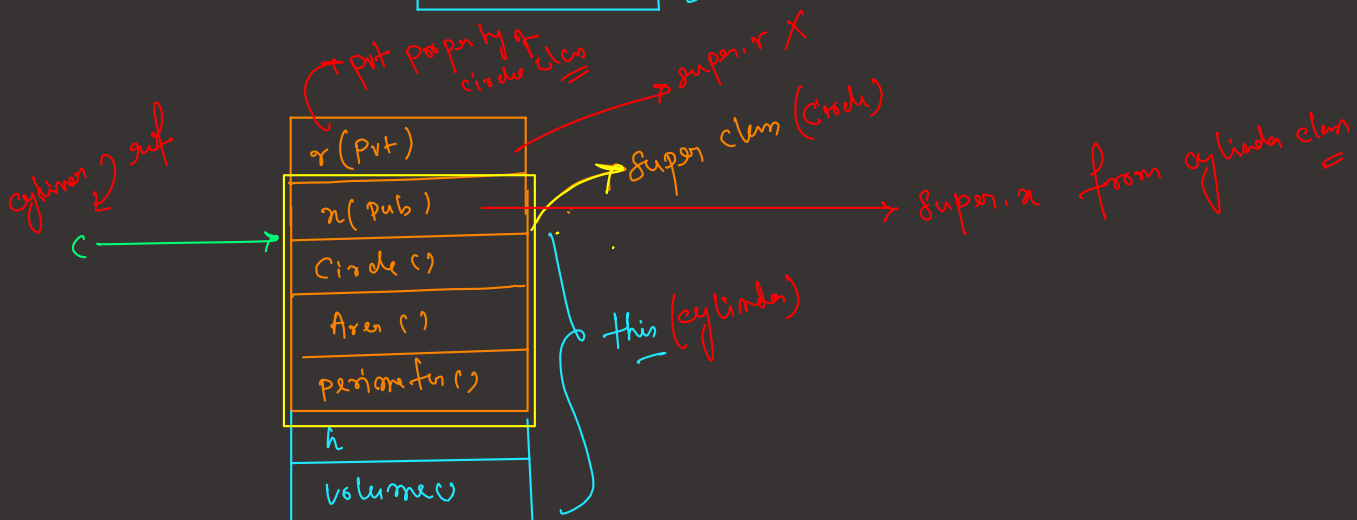
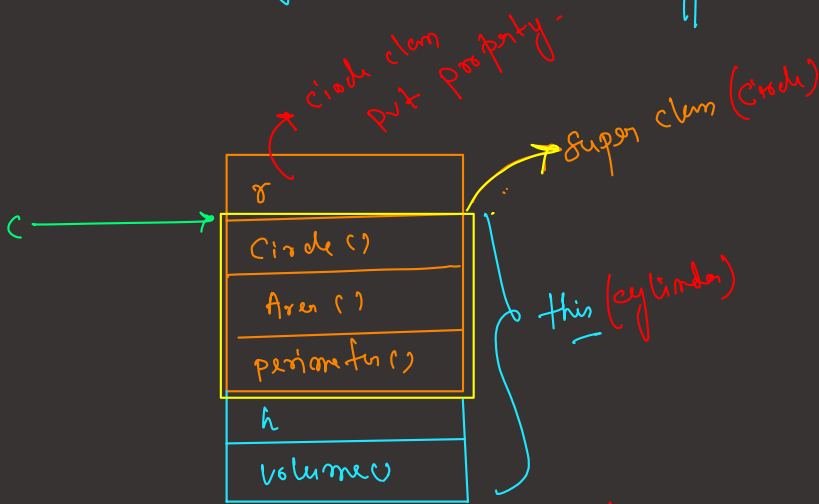
Circle
 ↑
 Cylinder

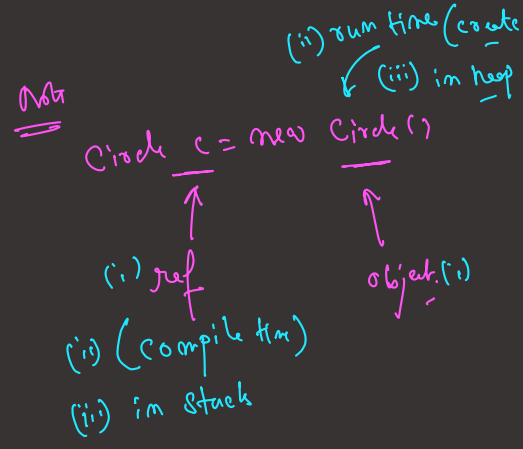
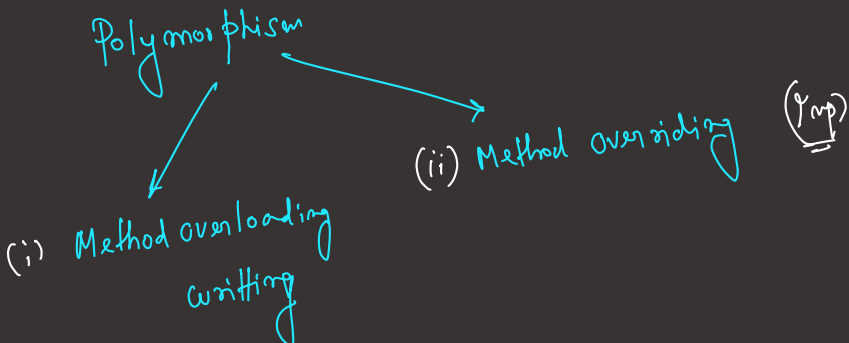
ref to the current object
 'this' keyword



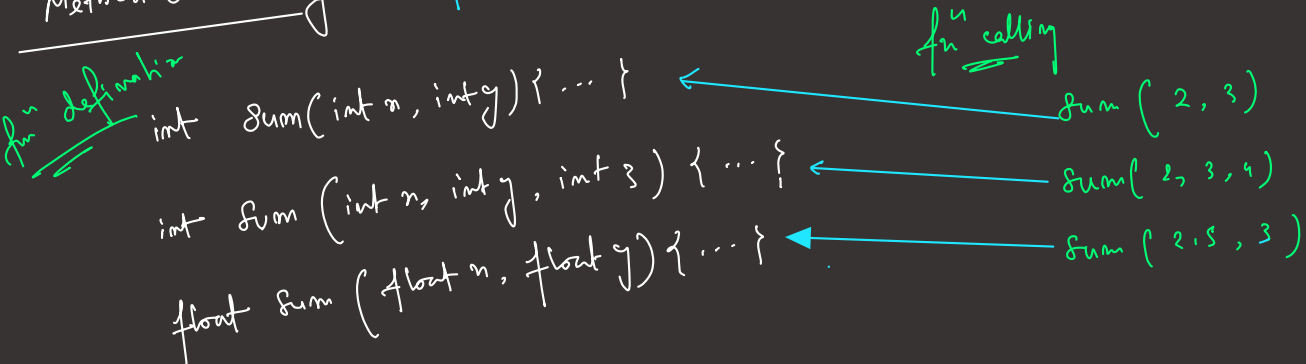
'super' keyword
 → ref. to the parent class object
 super class object

Cylinder c = new Cylinder(5, 10) → // super class constructor() call Circle()
 // current class constructor() call Cylinder()

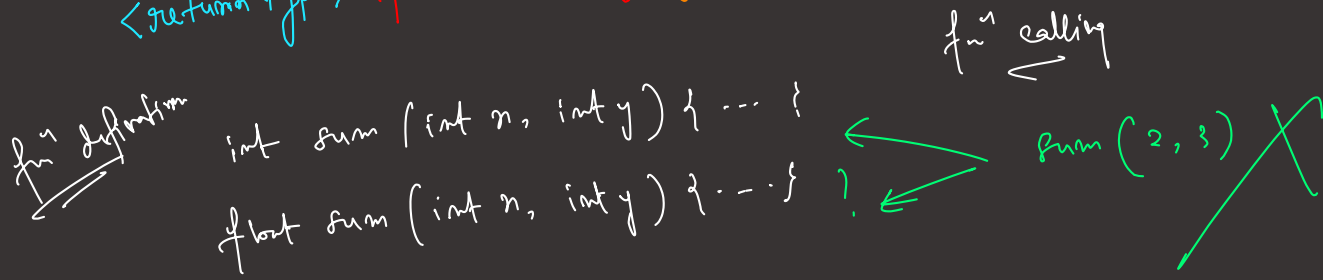




Method Overloading (compile time Polymorphism)



<return type> function name (type parameter list)



(ii) Overriding [run time] → inheritance only.

```

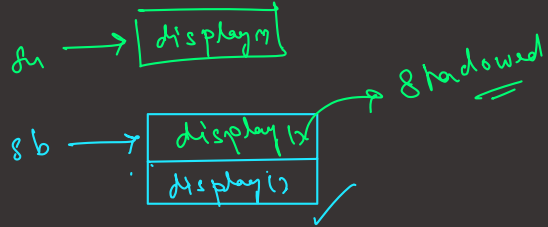
class Super {
    display() {
        s.o.p("Super Hello");
    }
}
  
```

```

class Sub extends Super {
    display() {
        S.o.p("Sub Hello");
    }
}
  
```

```

Super su = new Super();
Sub sb = new Sub();
  
```



for $\rightarrow$

display 1
display 2

8b →

display 1
display 2
display 2
display 3

→ shadowed

→ overridden method

dynamic Method Dispatch

sub. class obj.

super class ref.

display1

display2

display3

override method

$S_{\text{display}}^1() \rightarrow \text{Super display}^1$
 $S_{\text{display}}^2() \rightarrow \text{Sub display}^2$
 $S_{\text{display}}^3() \rightarrow X$

